AVL Tree

Height balanced binary search tree.

Each node has a balance factor between -1 and +1.

Balance factor (B.F.) = $h_L - h_R$

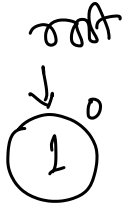
h_L → height of left subtree

h_R → height of right subtree.

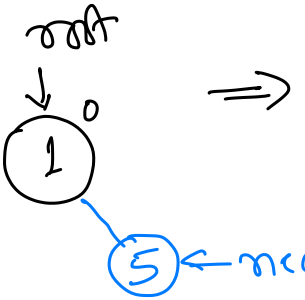
insert (1)

root → empty

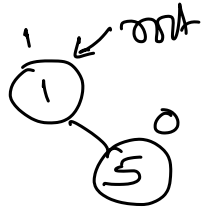
⇒

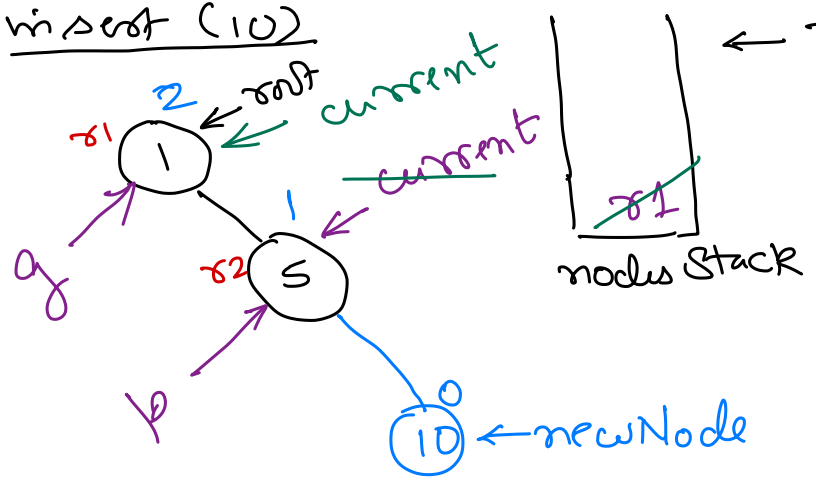


insert (5)



⇒





← To track nodes in path from root till newNode.

Left Rotation

RR Imbalance

g → nearest parent of newNode, having incorrect balance factor.

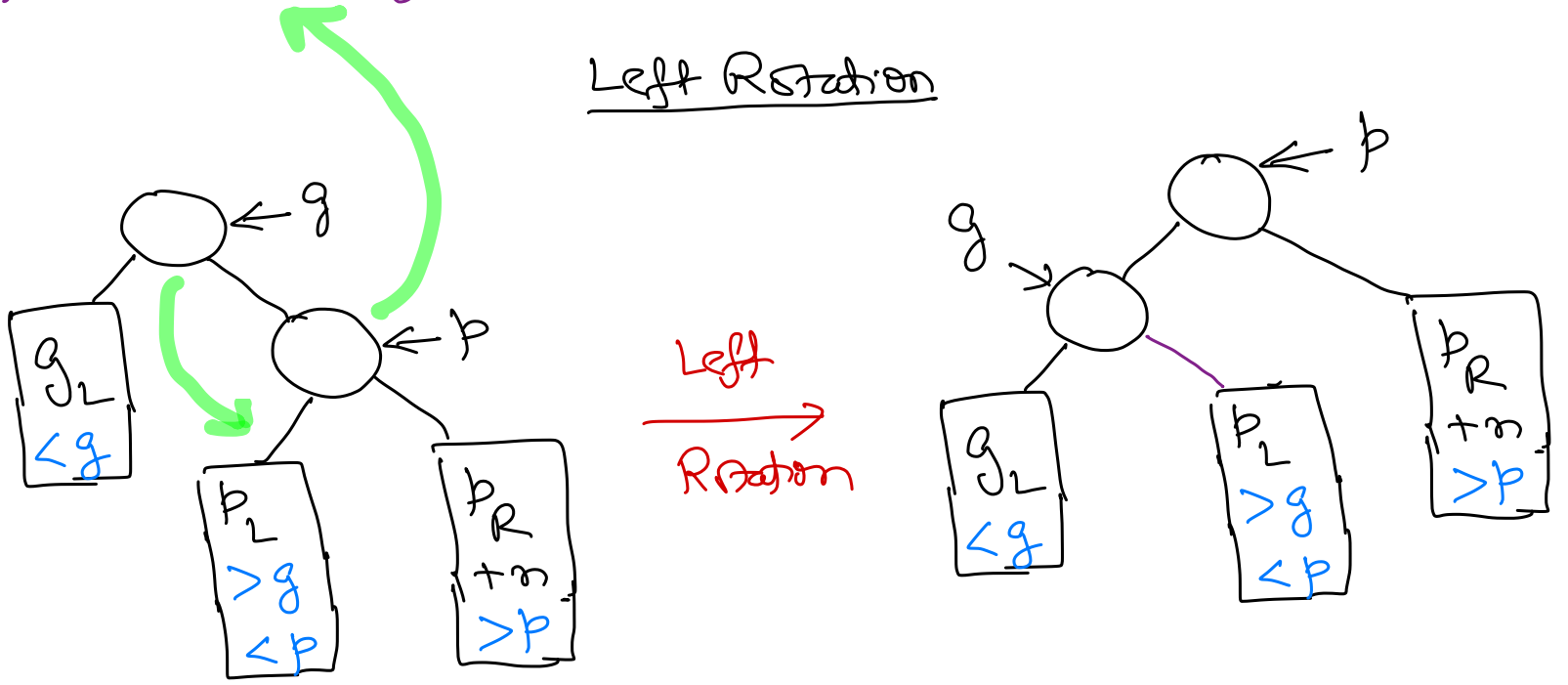
Take two steps from g towards newNode (we may reach newNode, we might not – doesn't matter).

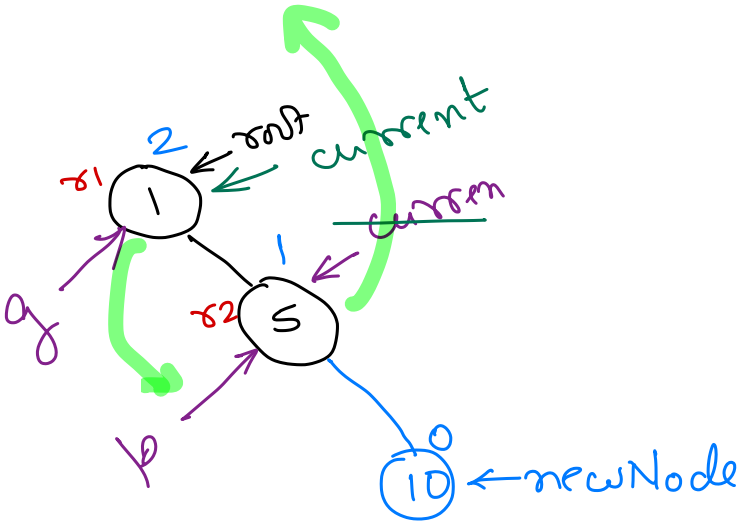
Record L, if followed left child.

Record R, if followed right child.

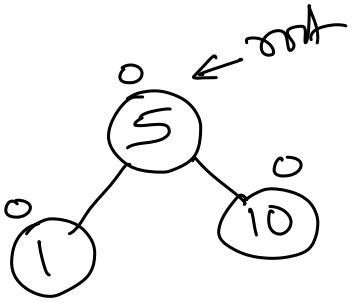
For Left or Right rotation, p is child node of g, in the path from g to new Node.

Left and Right rotations are performed around p.

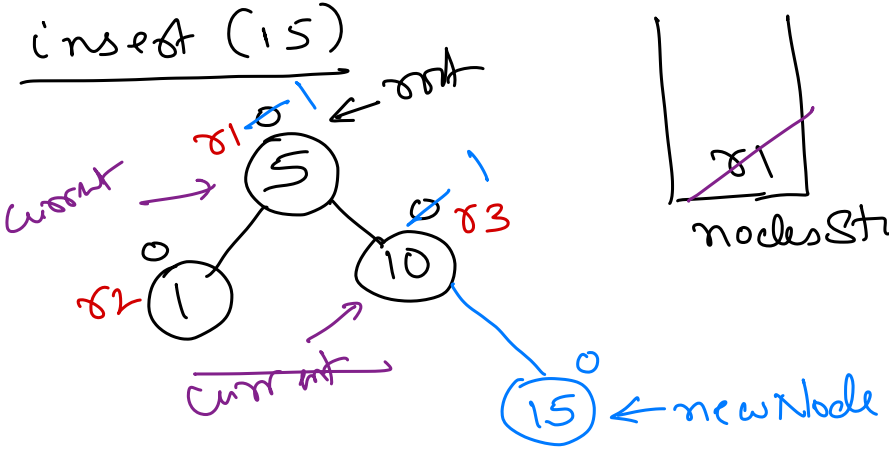




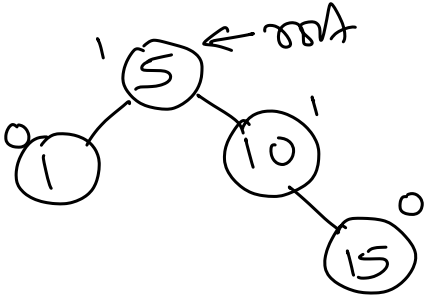
Left Rotation →



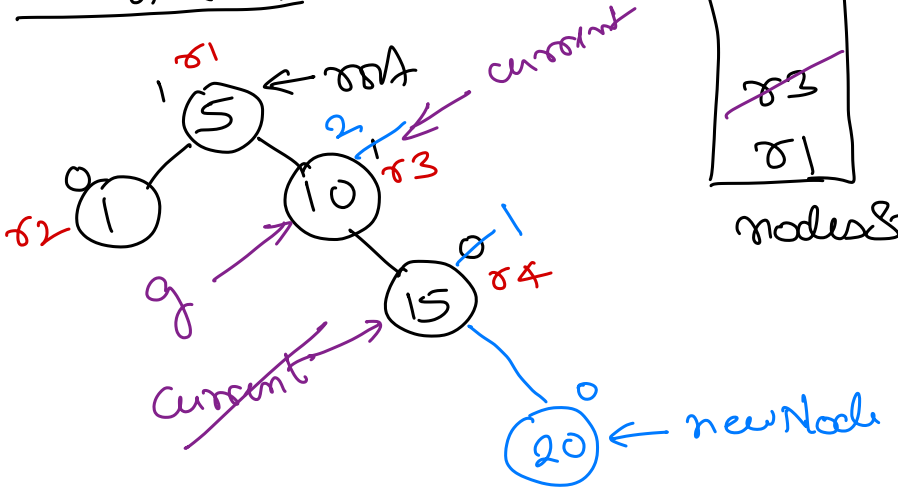
insert (15)



⇒



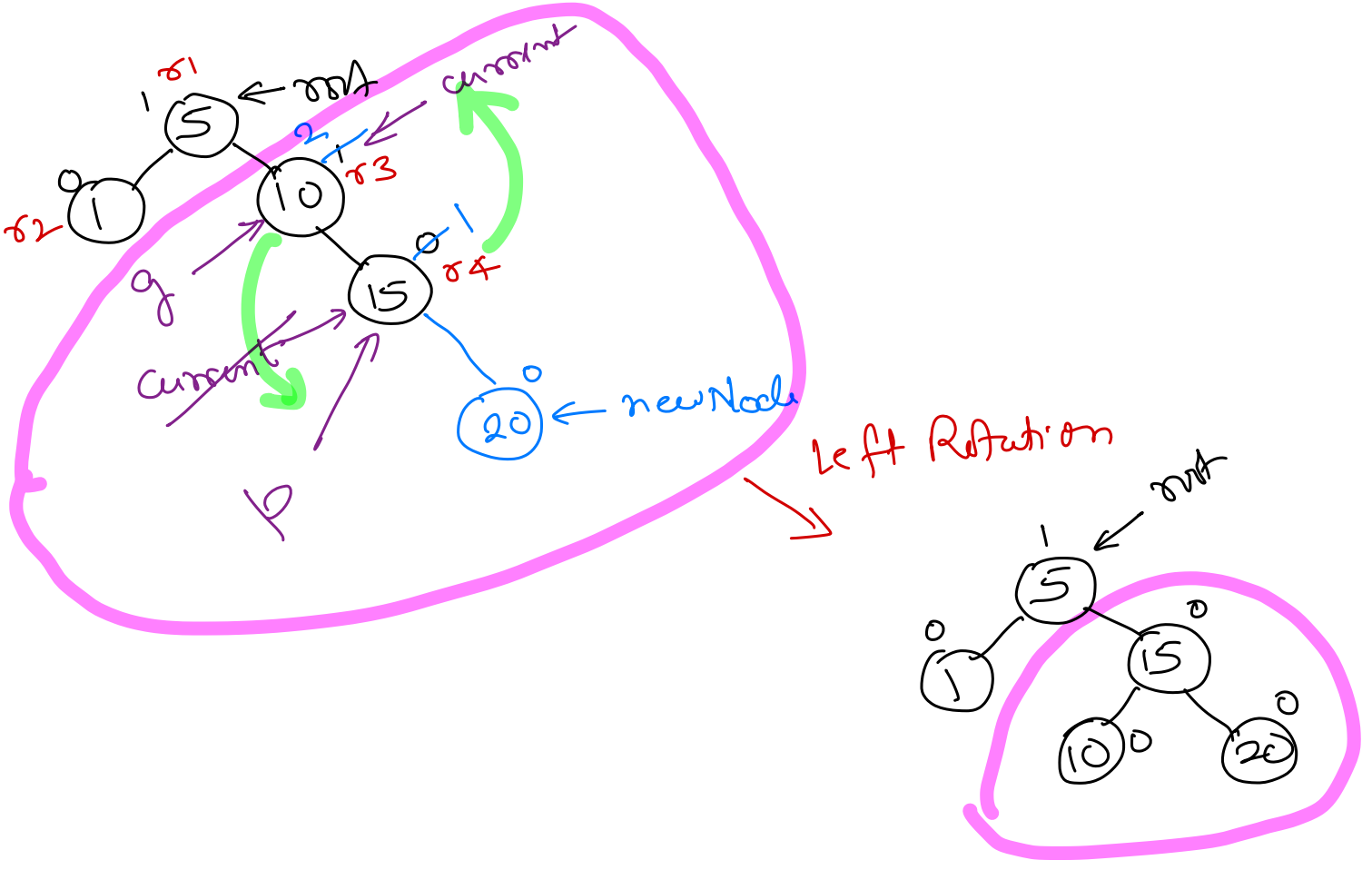
insert (20)

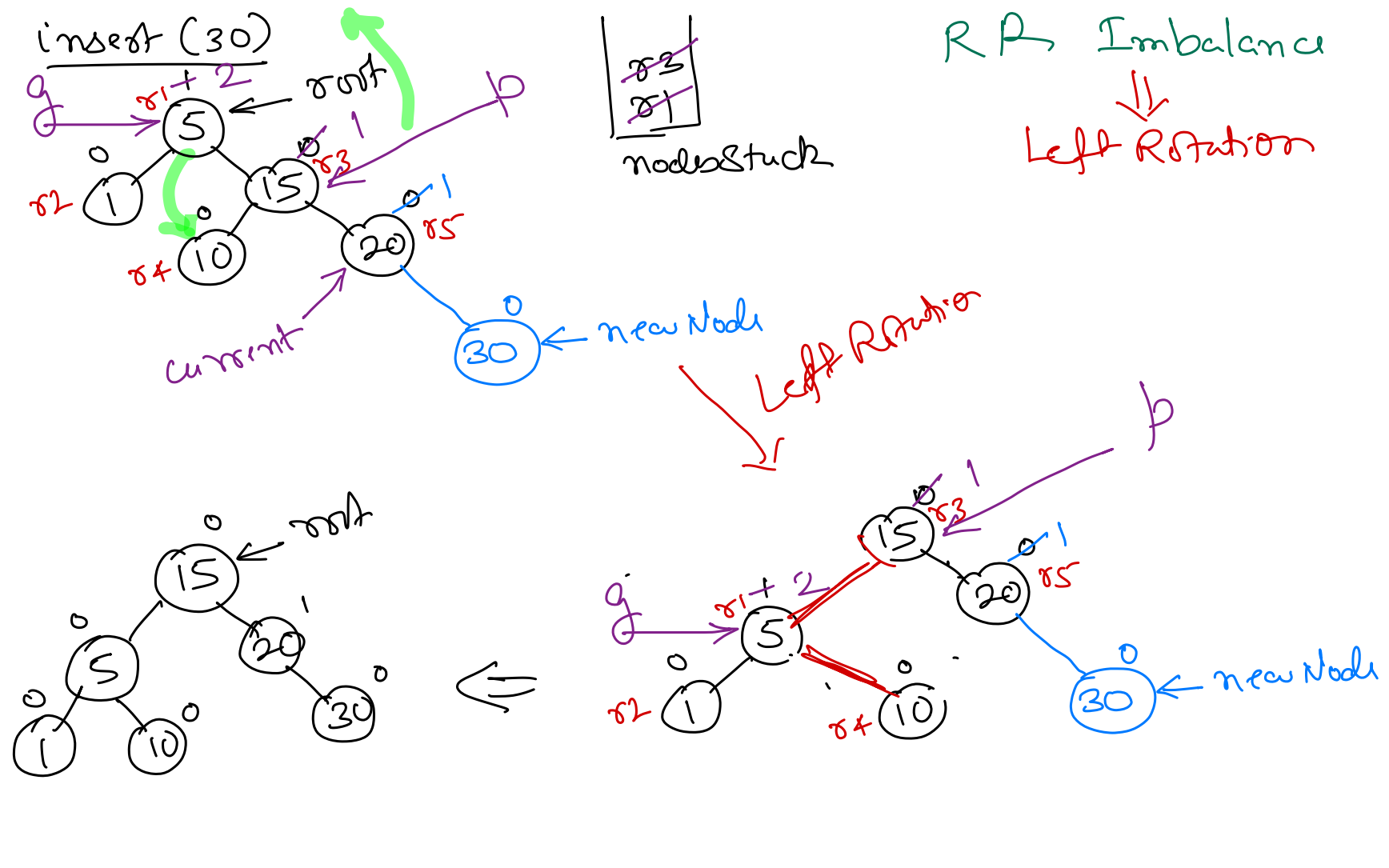


RR Imbalance

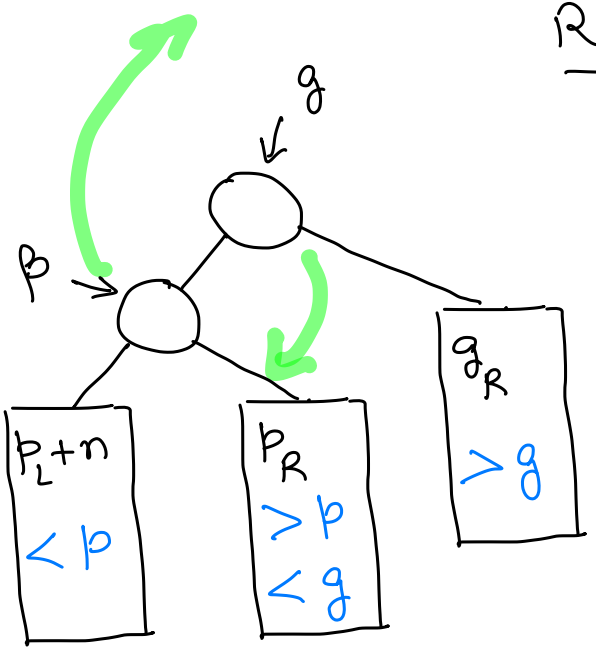
⇓

Left Rotation

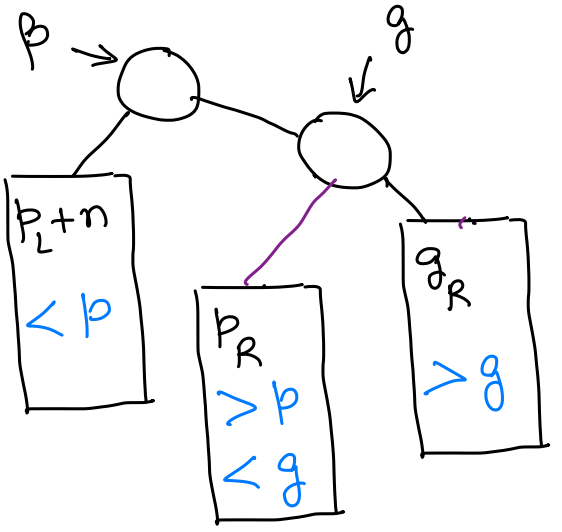

r1
root
5
1
r2
1
2
10
r3
current
q
15
r4
current
p
20
newNode
Left Rotation
root
5
1
15
1
10
20


insert (30)
g
r1 + 2
root
p
5
1
15
20
10
30
r2
r3
r4
r5
current
newNode
nodesStuck
RR Imbalance
Left Rotation
Left Rotation
root
15
5
20
1
10
30
g
r1 + 2
p
r3
r5
r2
r4
newNode

Right Rotation → Mirror image of Left Rotation.

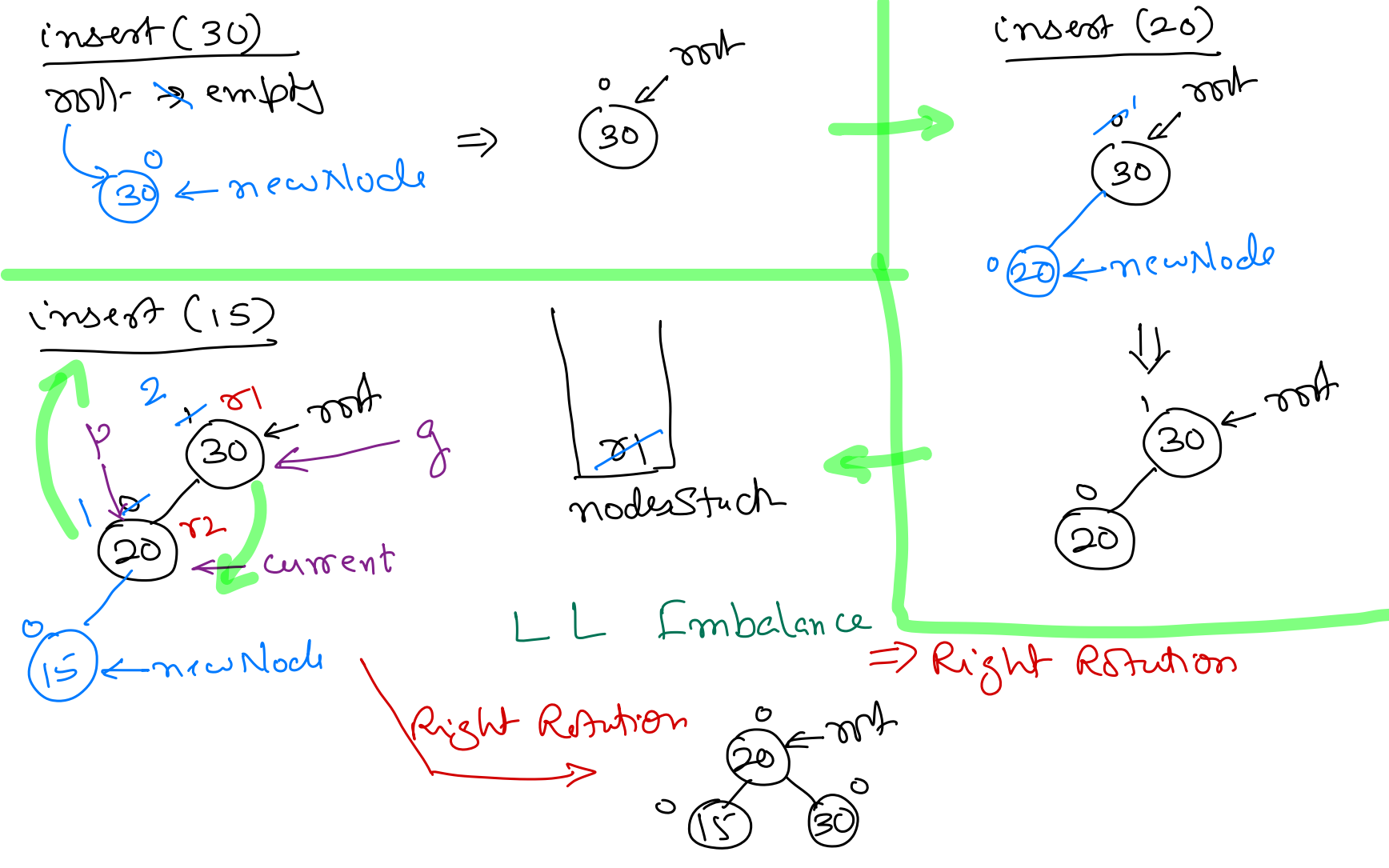


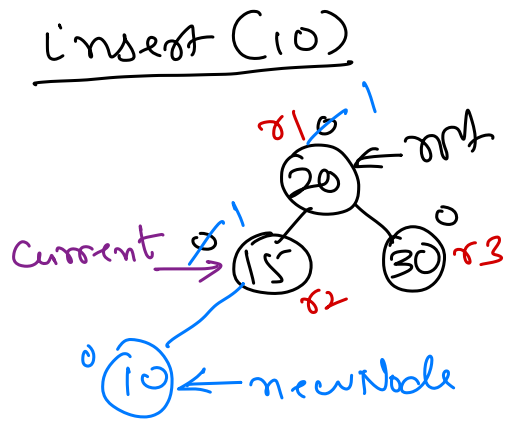
Right Rotation →

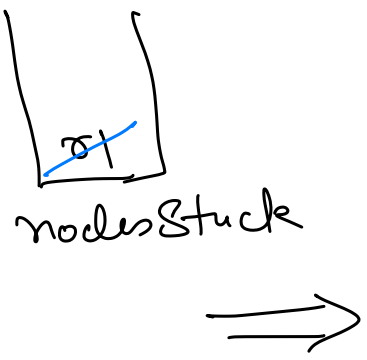


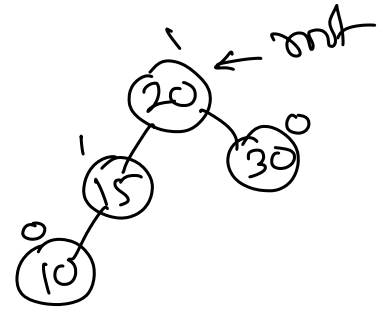
---

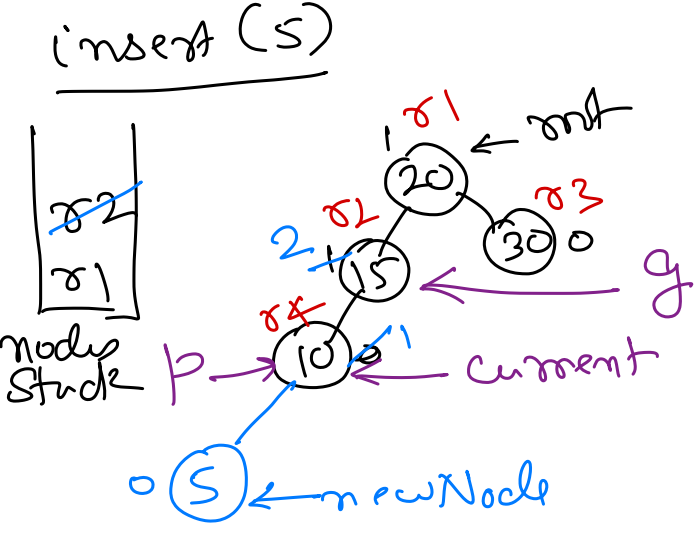
insert : 30 20 15 10 5 1


insert (30)
root -> empty
30
newNode
root
insert (20)
30
20
newNode
insert (15)
2
p
1
r1
30
root
g
20
r2
current
15
newNode
r1
nodesStuck
1
30
20
LL Imbalance
Right Rotation
Right Rotation
20
15
30





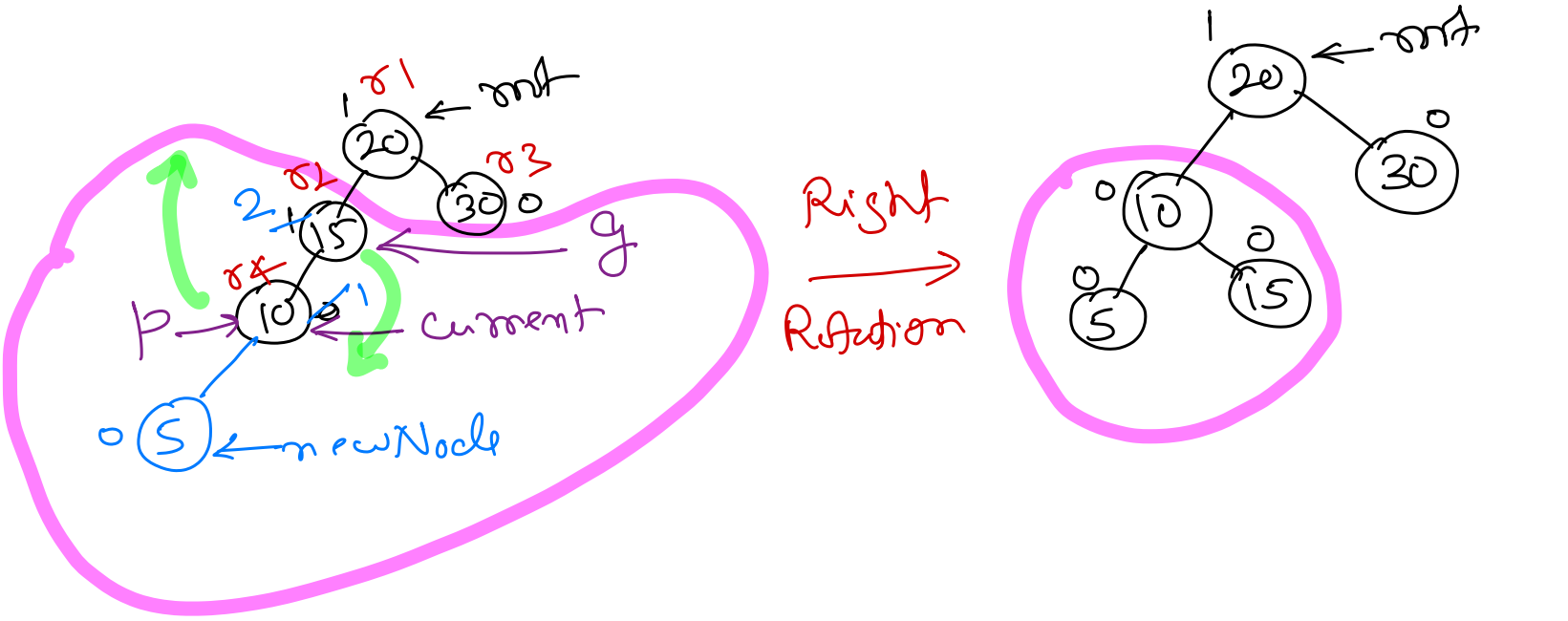


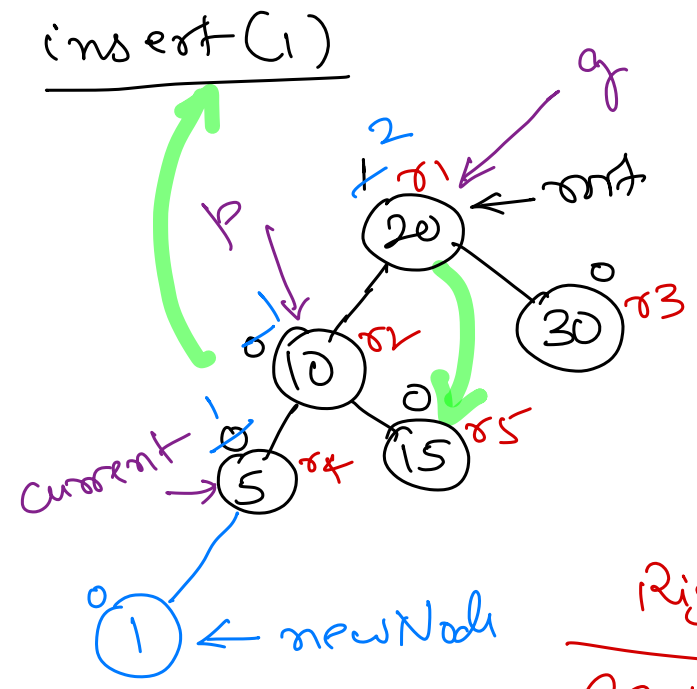


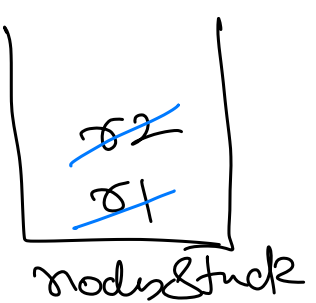
LL Imbalance

⇓

Right Rotation

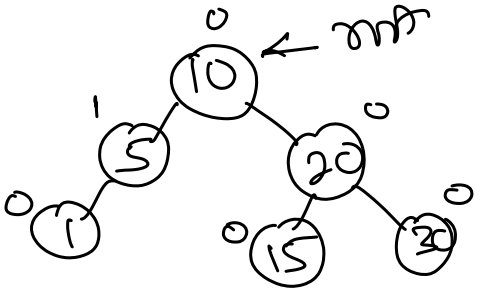

r1
r2
r3
r4
20
30
15
10
5
p
g
current
newNode
Right
Rotation

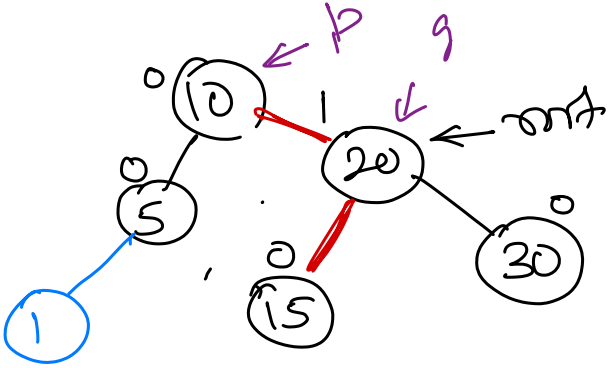




LL Imbalance ⇒ Right Rotation

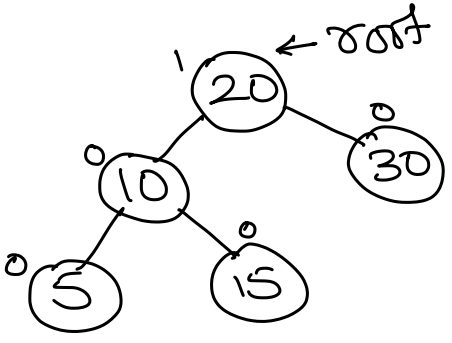
Right Rotation



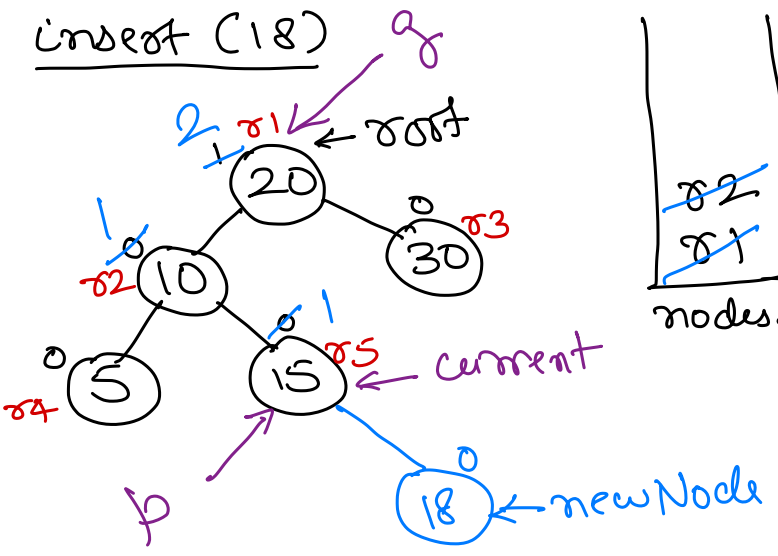


Insert :

30 20 15 10 5



insert (18)



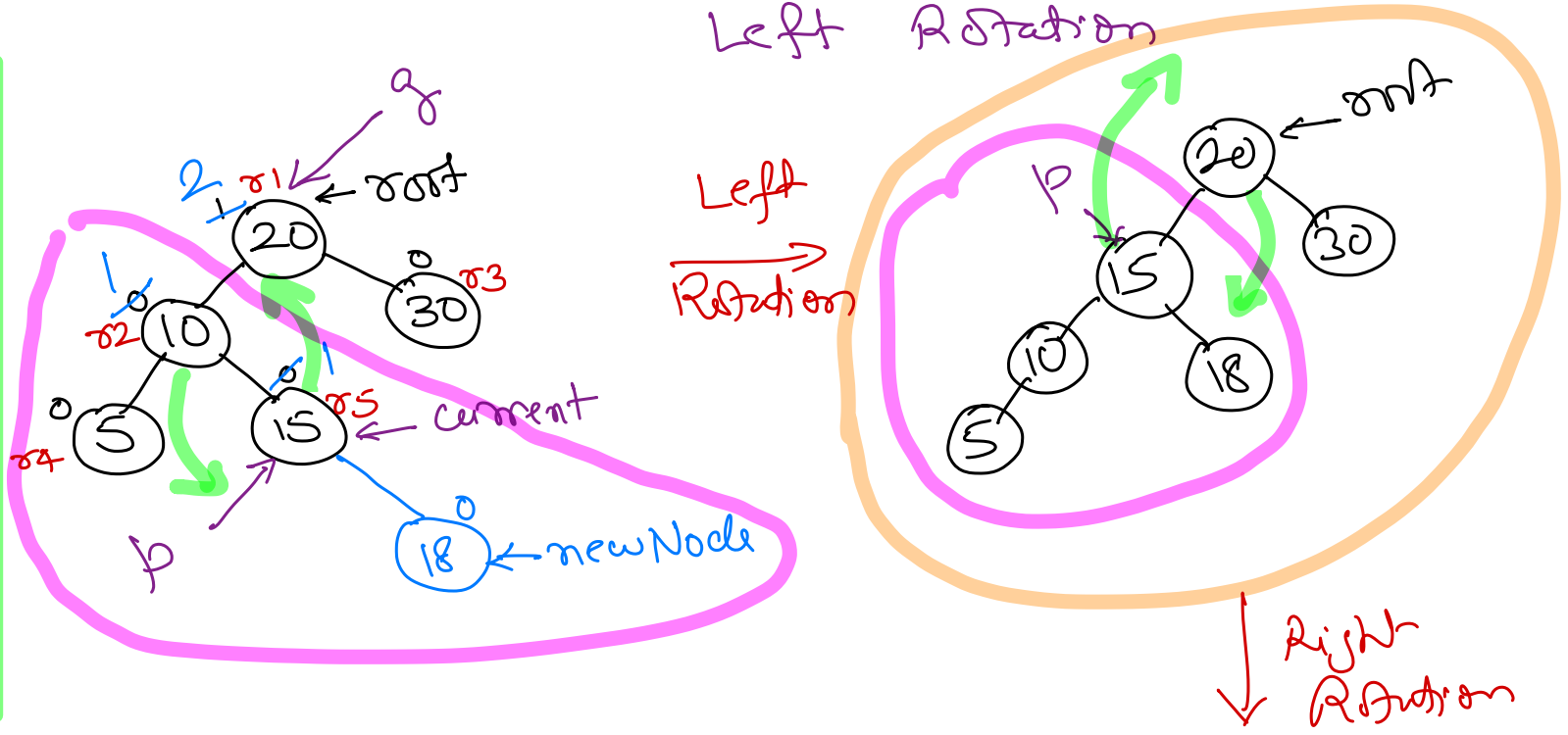
LR Imbalance

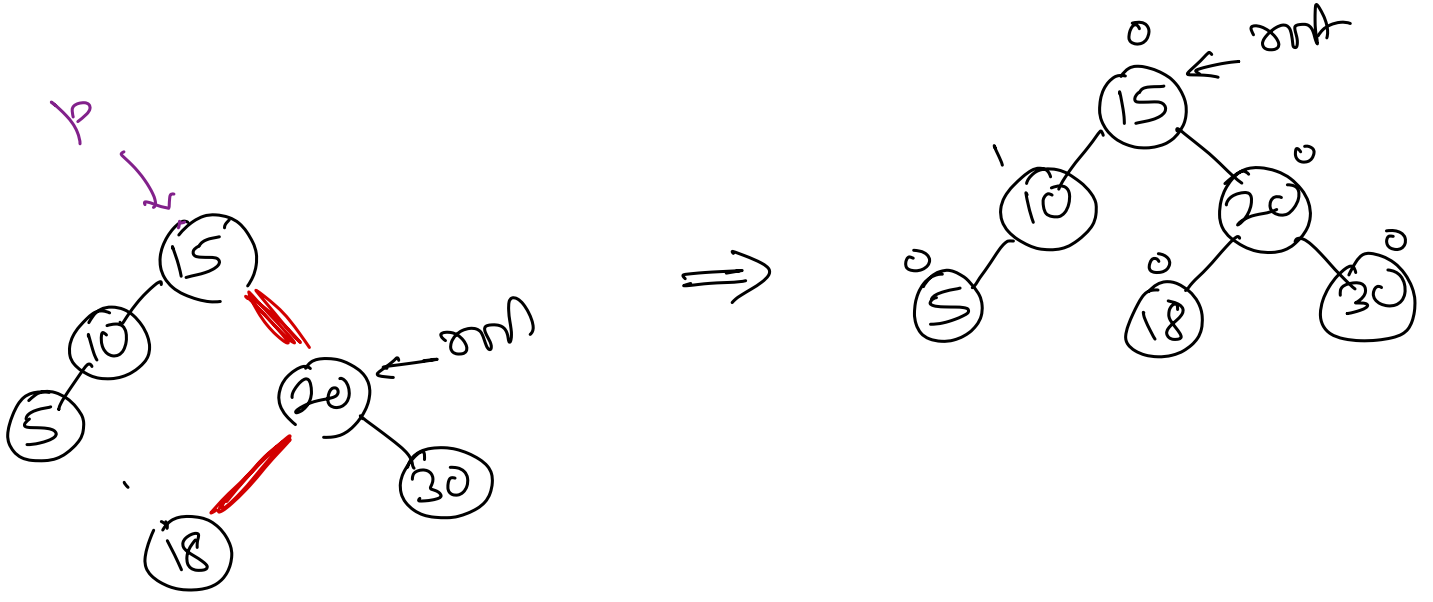
For LR imbalance and RL imbalance, we perform two rotations around p.

p → grand child of g, in path from g towards new Node.

LR imbalance ⇒ Left Rotation
Right Rotation

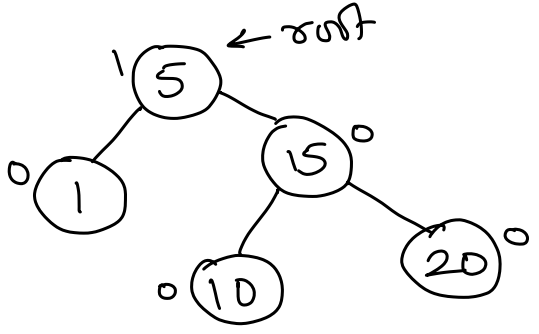
RL imbalance ⇒ Right Rotation
Left Rotation



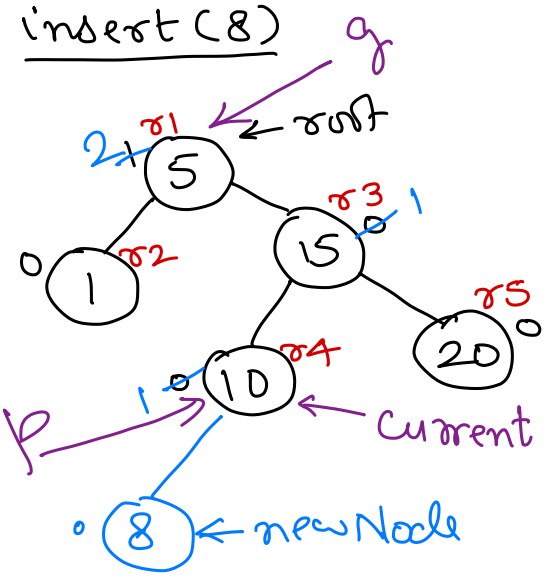

p
15
10
20
5
18
30
null
⟹
0
15
1
10
0
20
0
5
0
18
0
30
null

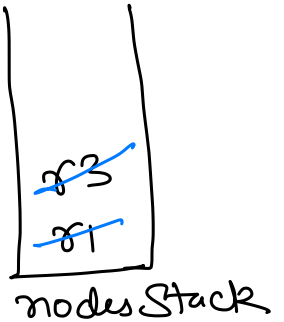
Insert:

1 5 10 15 20



insert(8)

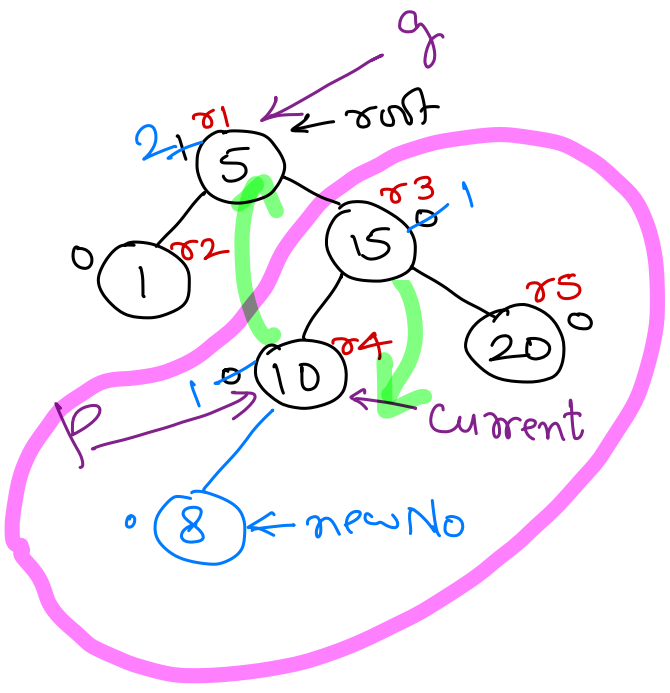




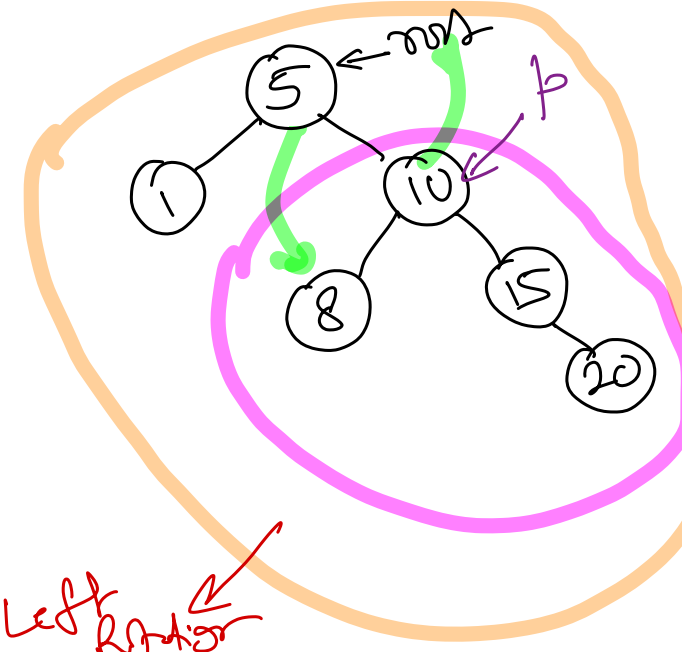
RL Imbalance

⇓

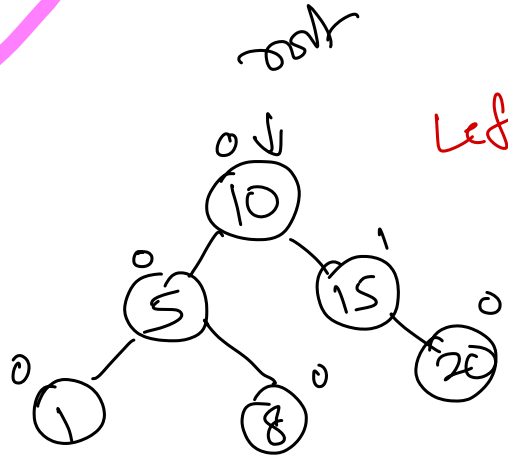
Right Left Rotations



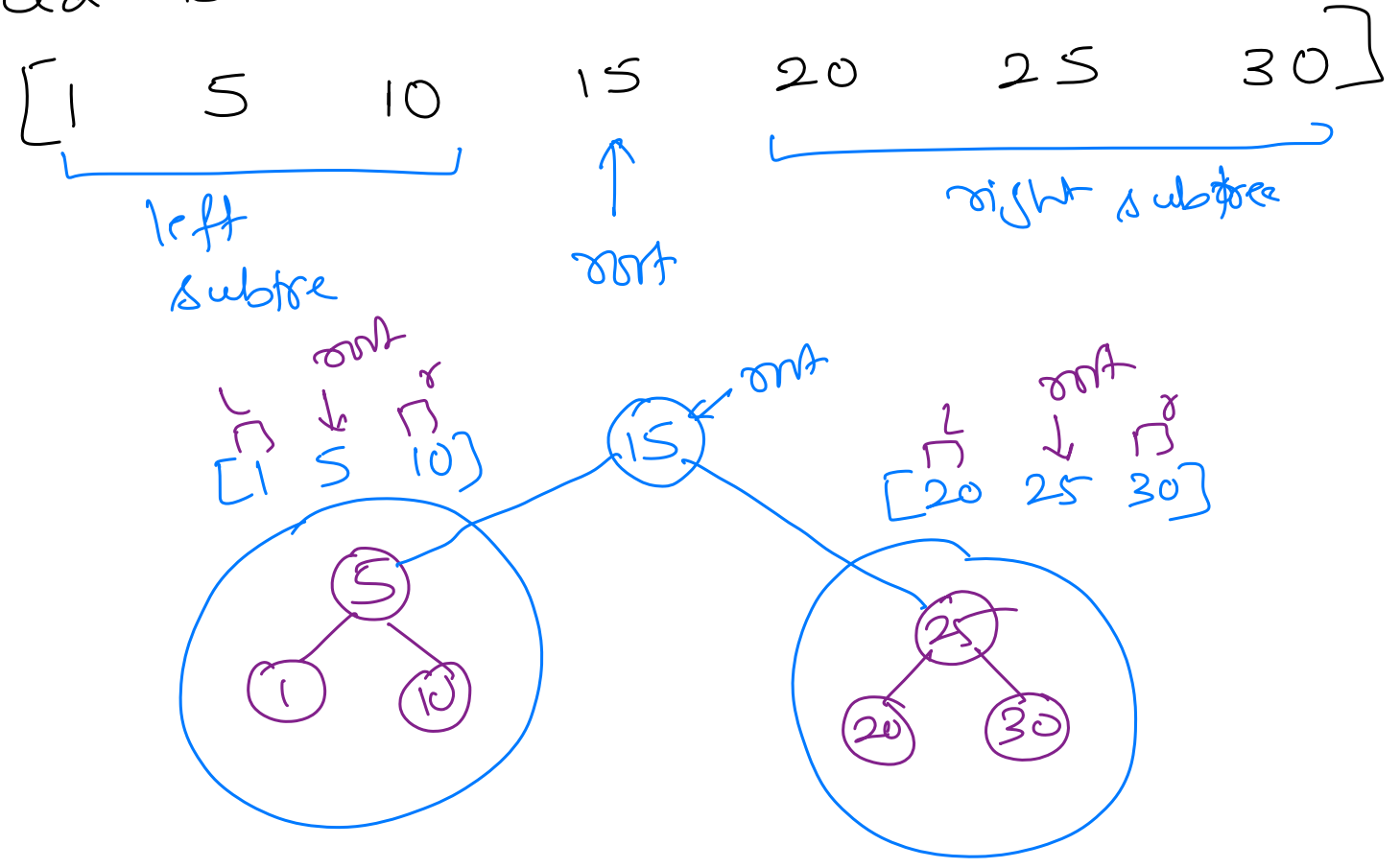
Right Rotation



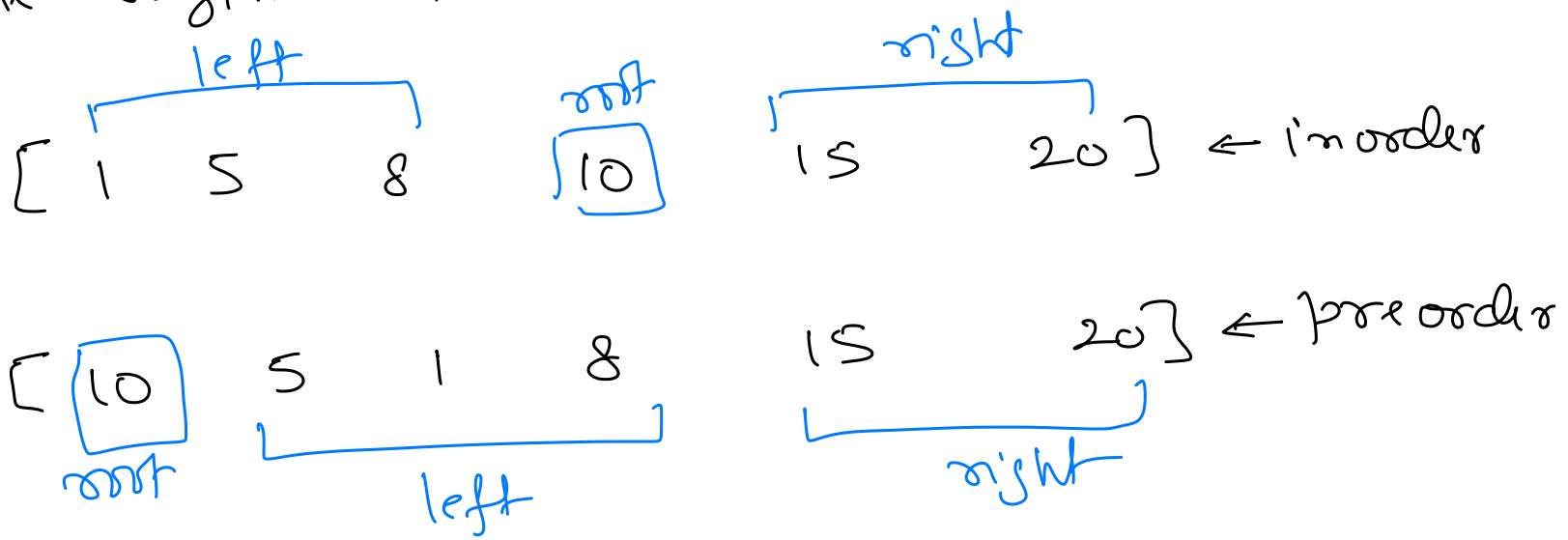
Left Rotation



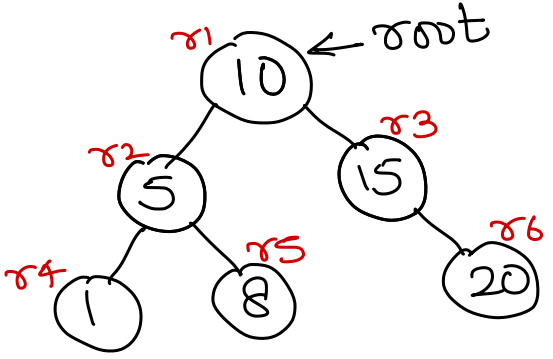
Given a list of sorted elements, create a balanced BST.

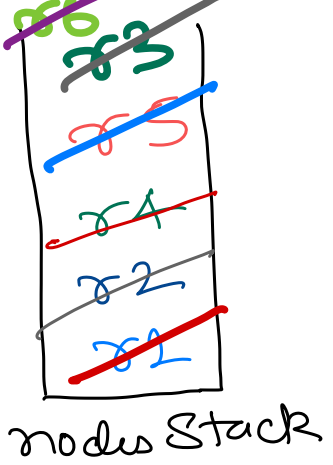


Given two traversals of a binary tree. Create the original tree back.



Iterative inorder traversal





Current → ~~x1~~ ~~x2~~ ~~x4~~ ~~null~~ ~~x4~~ ~~null~~ ~~x2~~ ~~x5~~ ~~null~~ ~~x5~~ ~~null~~ ~~x1~~ ~~x3~~ ~~null~~ ~~x3~~ ~~x6~~ ~~null~~ ~~x6~~ ~~null~~

O/P: 1 5 8 10 15 20

Iterative inorder

Time complexity = O(n)
Space complexity = O(log n)

- current = root
- while ((current != null) || (!nodesStack.empty()))
  - while (current != null)
    - Push current on nodesStack
    - current = current's left child.
  - current = Pop a node from nodes stack.
  - Process current node
  - Current = current's right child.

space used by nodes stack.

InOrder(root)
- if (root is empty) then
  - Stop.
- If (root node's left child exists) then
  - InOrder(root's left child).
- Process root node's data.
- If (root node's right child exists) then
  - InOrder(root's right child).
- Stop.

Space Complexity

Extra space taken by algorithm.

↓

to store additional data.

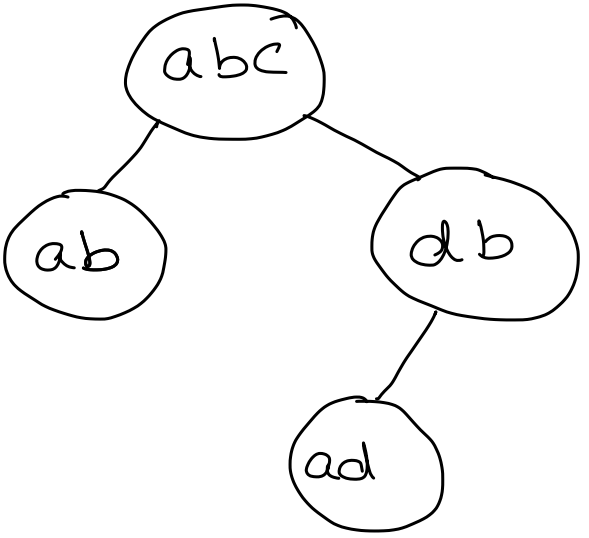
Time complexity = $O(n)$

Space complexity = ~~$O(1)$~~

=

$O(\log_2 n)$ ← space taken by recursive calls on system stack.

Dictionary of words : abc db ab ad



Using BST

w → length of a word

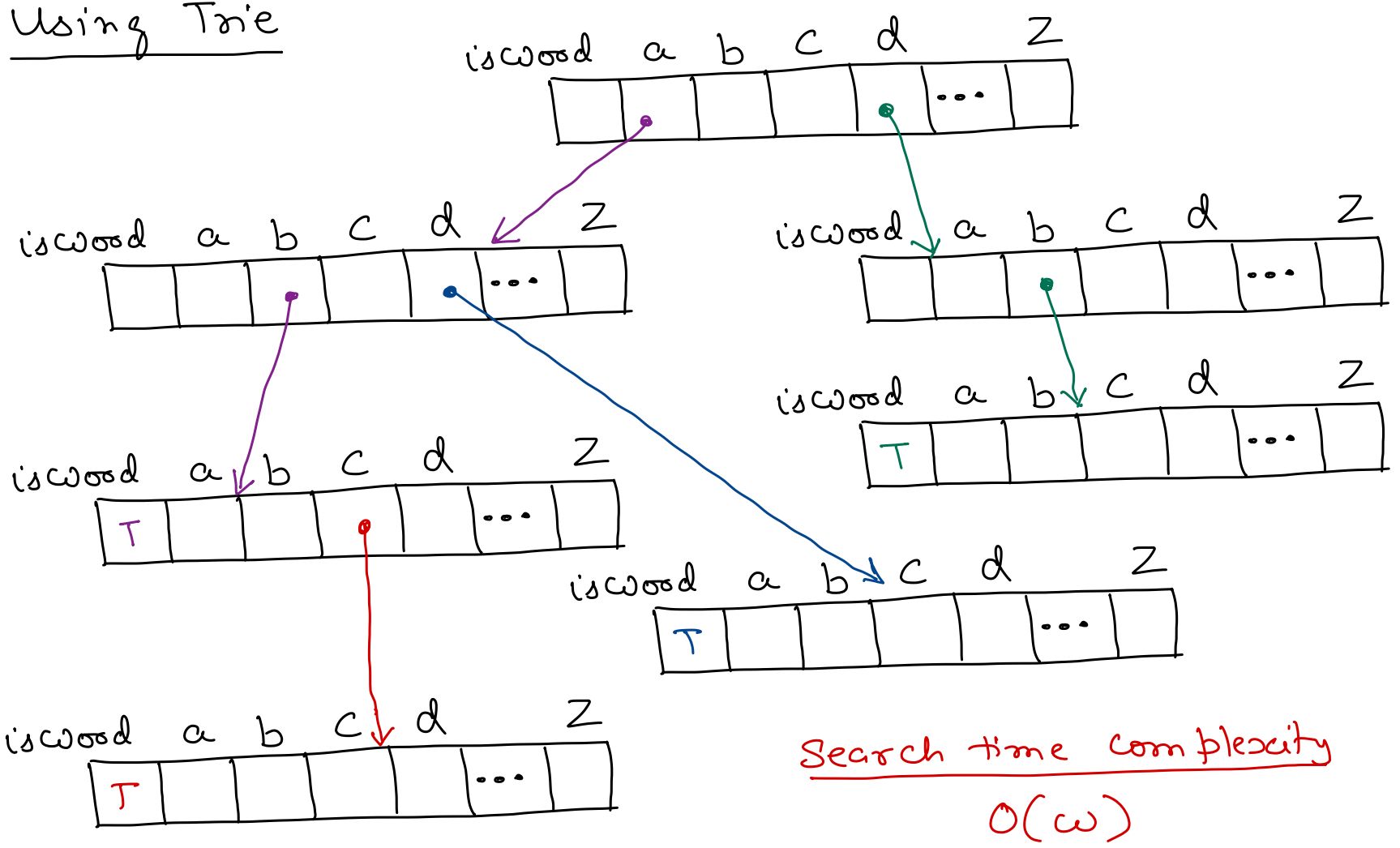
Search time complexity

$O(w \log_2 n)$

---

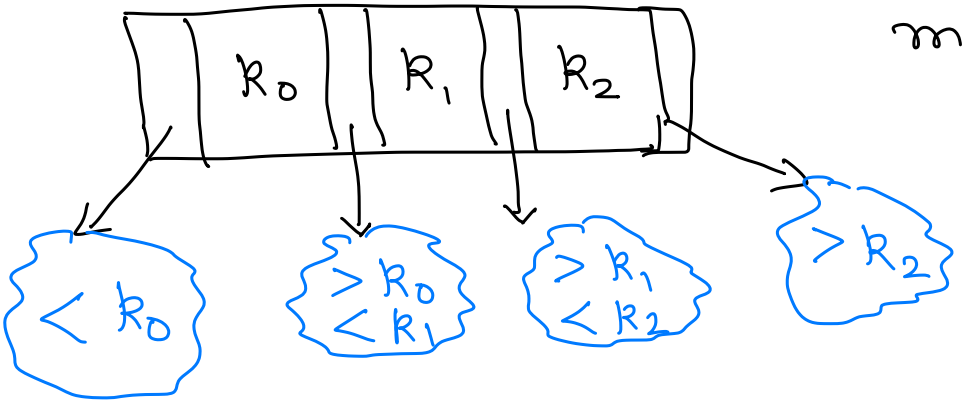
ab abc ad db

Using Trie



m-way Search Tree → B-Tree family.

B-Tree of order m, each node has at most m children.



$m = 4$.

A B^+-Tree with $m = 4$.